

MSc in Mathematical and Computational Finance

Deep Learning

University of Oxford - Candidate Number: 1100599

HT26

1(a) Training and test data generation for the Heston pricing map

We generate supervised pairs $(x_i, P(x_i))$, $x = (S_0, Y_0, \kappa, \mu, \sigma, r, \rho, K, T)$, $P(x) = \mathbb{E}[e^{-rT}(S_T - K)^+]$, under the Heston model. The input parameters are sampled uniformly over the ranges

$$S_0 \in [80, 120], \quad Y_0 \in [0.01, 0.35], \quad \kappa \in [0.5, 5], \mu \in [0.01, 0.25], \quad \sigma \in [0.1, 1], \\ r \in [0, 0.1], \quad \rho \in [-0.95, 0], \quad K \in [60, 140], \quad T \in [0.1, 2].$$

These intervals are chosen to cover realistic **equity** option configurations. In particular, negative values of ρ reflect the empirically negative correlation between returns and volatility. The ranges for κ, μ, σ span a wide variety of variance regimes, while the ranges for S_0, K, T cover different levels of moneyness and maturities.

To avoid pathological variance dynamics, we enforce the Feller condition $2\kappa\mu > \sigma^2$ by rejection sampling. For each sampled x , the price $P(x)$ is approximated by Monte Carlo simulation. We use a full truncation Euler scheme for the variance process, $Y_n^+ = \max(Y_n, 0)$, $Y_{n+1} = Y_n + \kappa(\mu - Y_n^+)\Delta t + \sigma\sqrt{Y_n^+}\sqrt{\Delta t}\tilde{Z}_n^2$, where $\tilde{Z}_n^2 = \rho Z_n^1 + \sqrt{1 - \rho^2} Z_n^2$, $Z_n^1, Z_n^2 \sim \mathcal{N}(0, 1)$.

For the stock price, we use exponential Euler update: $S_{n+1} = S_n \exp\left[\left(r - \frac{1}{2}Y_n^+\right)\Delta t + \sqrt{Y_n^+}\sqrt{\Delta t}Z_n^1\right]$, which preserves the positivity of S_t , while full truncation prevents numerical issues due to negative variance values. To reduce Monte Carlo variance, we implement antithetic variates by generating Gaussian increments (Z_1, Z_2) and completing the sample with their opposites $(-Z_1, -Z_2)$, and averaging the corresponding payoffs. The Monte Carlo estimator is $\hat{P}_M(x) = \frac{1}{M} \sum_{m=1}^M e^{-rT}(S_T^{(m)} - K)^+$.

Simulations are fully vectorized and parallelized on GPU. We use $N_{\text{steps}} = 200$ time steps per path and generate $N_{\text{train}} = 100,000$, $N_{\text{test}} = 20,000$, with $M_{\text{train}} = 40,000$, $M_{\text{test}} = 90,000$. The training set is larger because the neural network needs broad coverage of the 9-dimensional input space. The number of paths is larger in the test set to ensure a more accurate estimate of the true price, reducing Monte Carlo noise in the out-of-sample evaluation.

1(b) Neural network architecture, training procedure, and global accuracy

We model the pricing map $x \mapsto P(x)$ with a feed-forward multi-layer perceptron (MLP), where $x = (S_0, Y_0, \kappa, \mu, \sigma, r, \rho, K, T)$ is a 9-dimensional input and the output is a scalar option price. Input features are standardized using training-set mean and standard deviation, as they lie on different scales (e.g. $S_0, K \sim 10^2$ vs. r, μ, Y_0). The architecture is a fully connected MLP with four hidden layers of widths $256 \rightarrow 256 \rightarrow 128 \rightarrow 64$, with ReLU activations, batch normalization, and dropout (rate 0.05). Batch normalization stabilizes training, while dropout mitigates overfitting. We use Kaiming-uniform initialization.

Overfitting is further controlled via L^2 regularization (Adam weight decay 10^{-5}) and early stopping. The training set is split into 90% training and 10% validation. Model selection is performed on the validation set, and the test set is used only once for final evaluation.

Training uses mini-batches of size 1024 with Adam (learning rate 10^{-3}). A scheduler halves the learning rate when validation loss plateaus, and training stops if no improvement is observed for 12 epochs.

As the requested evaluation metrics are based on absolute errors, we chose a loss aligned with this objective. Direct MAE optimization is possible, but the absolute value is non-smooth at zero. We therefore use the **Huber loss** with parameter $\delta = 0.5$, which behaves quadratically for small errors and linearly for large ones, providing a good compromise between optimization stability and robustness. We also tested a custom weighted Huber loss designed to downweight certain observations (low-price calls), but was not retained.

Table 1 reports the final test performance, showing that the network provides an accurate surrogate for the Heston pricing function.

Dataset	MAE	Mean relative error
Train	0.164	0.014
Val	0.166	0.014
Test	0.155	0.013

Table 1: Global pricing accuracy of the final neural network.

Relative errors are unstable for near-zero prices. We therefore report MAE on the full dataset, and mean relative error only for $P > 0.5$, avoiding distortions from deep OTM options.

Distribution of the error. Figure 1 shows that errors are concentrated near zero, with a moderate right tail, indicating that large pricing errors are rare and overall accurate approximation of the Heston pricing map. The filtered relative-error distribution is also concentrated on small values, confirming good proportional accuracy once near-zero prices are excluded.

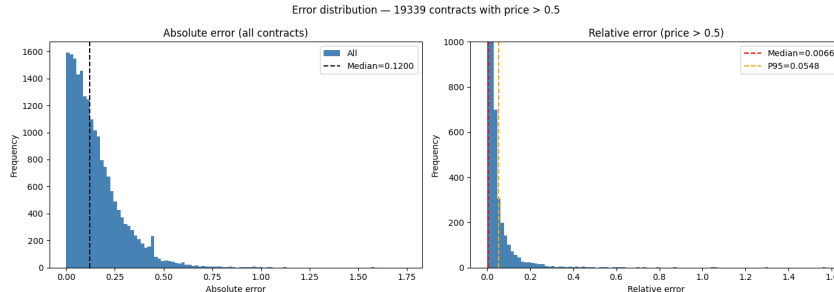


Figure 1: Distribution of the pricing errors on the test set. Left: absolute error on full test set. Right: relative error on the subset of contracts with price $P > 0.5$ (vertical scale truncated for readability).

The signed prediction error is approximately centered around zero, indicating that the model exhibits no significant bias. This is consistent with the predicted vs true price plots, where points lie close to the diagonal, showing accurate fit and good out-of-sample generalization.

Error analysis across contract types and parameter regions. We analyze errors as a function of moneyness S_0/K . Figure 2 shows that out-of-the-money (OTM) contracts exhibit higher relative errors than at-the-money (ATM) and in-the-money (ITM), even after filtering small prices. However, absolute errors are of similar magnitude across contract types, indicating that OTM options are mainly more sensitive in relative terms due to their lower prices.

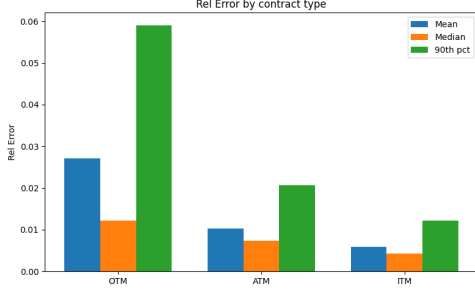


Figure 2: Relative error (subset $P > 0.5$) across contract types: OTM, ATM, and ITM.

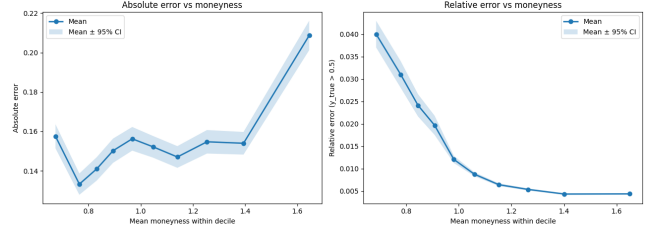


Figure 3: Absolute (left) and relative (right, filtered) errors as a function of moneyiness.

This is confirmed in Figure 3: relative error decreases with moneyiness, while absolute error remains relatively stable, with a slight increase for deep ITM options due to larger price scales. Overall, the model achieves consistent absolute accuracy, with relative errors driven primarily by price magnitude.

We also analyze the dependence of the error on other parameters, in particular the maturity T .

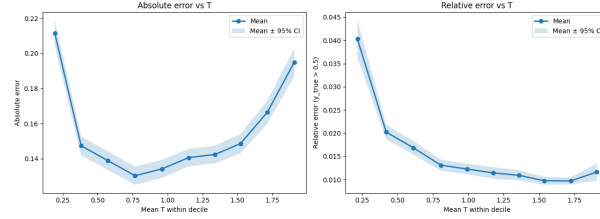


Figure 4: Absolute (left) and relative (right) errors as a function of maturity T .

Figure 4 shows that both short and long maturities are associated with larger absolute errors. Short maturities also exhibit higher relative errors due to smaller option prices, making the pricing function more sensitive. We also observe milder but consistent effects for parameters such as κ and μ : intermediate values are easier to approximate, while extreme values lead to larger errors. This suggests that regions corresponding to more complex or extreme volatility dynamics are more challenging for the neural network.

Impact of network capacity on accuracy. We study the effect of model capacity by increasing both depth and width (MLP with ReLU, trained with Adam), considering 2, 4, 6, and 8 hidden layers. Table 2 reports test performance (MAE and relative error). Increasing capacity from 2 to 6 layers improves performance, showing that a richer model better captures the pricing function. However, further increasing the capacity to 8 layers results in a degradation of performance on the test set. This suggests that the model becomes too complex for the task and starts to overfit, highlighting the existence of an optimal trade-off between model capacity and generalization.

Model (layers)	MAE	Relative Error
2 layers	0.2098	0.0184
4 layers	0.1554	0.0131
6 layers	0.1327	0.0104
8 layers	0.1549	0.0127

Table 2: Test set performance for different network capacities.

2. Calibration of SDE parameters using the neural-network approximator

(a) Generation of synthetic market data via Monte Carlo

To generate synthetic observed prices for the calibration step, we fix one set of Heston parameters and initial conditions,

$$S_0^* = 100, \quad Y_0^* = 0.04, \quad \kappa^* = 3.0, \quad \mu^* = 0.04, \quad \sigma^* = 0.4, \quad r^* = 0.02, \quad \rho^* = -0.7.$$

These values lie in the parameter domain introduced in Question 1 and correspond to a realistic equity-style stochastic volatility regime. They also satisfy the Feller condition, $2\kappa^*\mu^* = 0.24 > 0.16 = (\sigma^*)^2$, so that the variance process remains in a numerically well-behaved regime.

Keeping ν^* fixed, we generate option prices over a grid of strikes $K \in \{60, 65, 70, \dots, 140\}$ and maturities $T \in \{0.25, 0.50, 0.75, 1.00, 1.25, 1.50, 1.75, 2.00\}$. This gives $N = 8$ maturities and $M = 17$ strikes, hence a total of $N \times M = 136$ contracts.

For each pair (T_n, K_m) , we compute the corresponding call price $P_{n,m}$ using the same GPU Monte Carlo scheme as in Question 1(a). In order to obtain a sufficiently accurate synthetic price surface for calibration, we use 200,000 Monte Carlo paths and 1,000 time steps.

The resulting matrix $(P_{n,m})$ is used as the synthetic market dataset in the next part. The corresponding price surface is displayed in Appendix 6. As expected, call prices decrease with the strike and increase with maturity, producing a smooth surface suitable for calibration.

(b) Calibration via neural-network surrogate

We calibrate the Heston parameters $\nu = (\kappa, \mu, \sigma, r, \rho)$ by minimizing the MSE between neural network predictions and observed prices: $L(\nu) = \frac{1}{NM} \sum_{n=1}^N \sum_{m=1}^M (f(S_0^*, Y_0^*, \nu, K_m, T_n) - P_{n,m})^2$. The neural network weights are frozen, and only ν is optimized. To ensure that the parameters remain within the admissible domain, we optimize unconstrained variables mapped to bounds via sigmoid transformations. Optimization is performed using Adam. Although the problem is a nonlinear least-squares problem, and second-order methods (e.g. Gauss-Newton) could be more efficient, they require computing Jacobians of the network output and were not implemented. Since the objective is non-convex, we use a multi-start strategy: several initial points are sampled uniformly in the parameter domain, and optimization is run independently. The solution with lowest loss is retained. The multi-start strategy mitigates local minima and improves robustness. Hyperparameters are: learning rate 5×10^{-3} , 6000 iterations per run, 10 initializations, and gradient clipping at 1.0. These values were chosen to ensure stable convergence and were not extensively tuned.

Convergence. Figure 5 shows the evolution of the objective during optimization for the best run. The loss decreases rapidly and then stabilizes, indicating fast convergence. The best solution across all multi-start runs achieves $L(\nu) = 0.0263$.

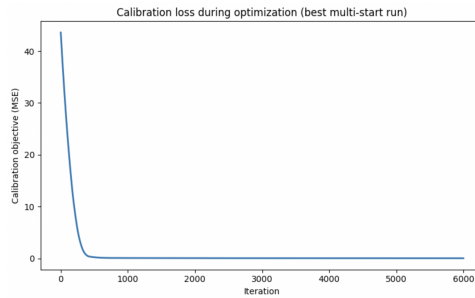


Figure 5: Calibration loss (MSE) during optimization.

(c) Accuracy of calibrated parameters

Parameter	True	Calibrated	Abs Error	Rel Error
κ	3.00	1.898	1.102	0.367
μ	0.04	0.050	0.010	0.253
σ	0.40	0.553	0.153	0.383
r	0.02	0.014	0.006	0.289
ρ	-0.70	-0.619	0.081	0.116

Table 3: True vs calibrated Heston parameters.

The calibrated parameters are reasonably close to the true values, with larger discrepancies for κ and σ . This reflects the ill-posed nature of the calibration problem: different parameter combinations can produce similar price surfaces.

Figure 7 (Appendix) shows that parameters converge quickly, but not necessarily to their true values, highlighting both non-convexity (multiple local minima) and limited identifiability. Moreover, since the neural network is only an approximation of the true pricing function, the true parameters ν^* do not necessarily minimize the surrogate loss.

Fit to the observed price surface. The calibrated model closely reproduces the observed price surface $P_{n,m}$ (see Appendix, Figures 8 and 10). Errors remain small across most of the (K, T) grid, with larger deviations mainly at extreme strikes and maturities. This shows that accurate pricing can be achieved despite discrepancies in parameter values.

Out-of-sample evaluation. To assess the generalization ability of the calibrated model, we generate a test dataset using maturities and strikes that were not included in the calibration grid. The test grid is defined as: $T \in \{0.35, 0.65, 0.95, 1.15, 1.40, 1.60, 1.85\}$, $K \in \{62.5, 67.5, \dots, 132.5\}$. These values differ from the training grid.

Dataset	MAE	RMSE	Mean Rel Error
In-sample	0.128	0.165	50.49
Out-of-sample	0.11	0.138	1.95

Table 4: Pricing errors for the calibrated model.

Figure 9 & 8 shows predicted versus true Monte Carlo prices on the test set, with points closely aligned along the diagonal, indicating an excellent fit both in- and out-of-sample. Interestingly, the relative error is significantly lower out-of-sample. This is mainly due to the fact that the test grid avoids the most extreme strike values, where option prices are very small and relative errors tend to explode due to division by near-zero values (see Section 1(b)). These results confirm that the calibrated model generalizes well to unseen contracts, and that the neural network surrogate provides a robust approximation of the pricing function.

A Generated price surface

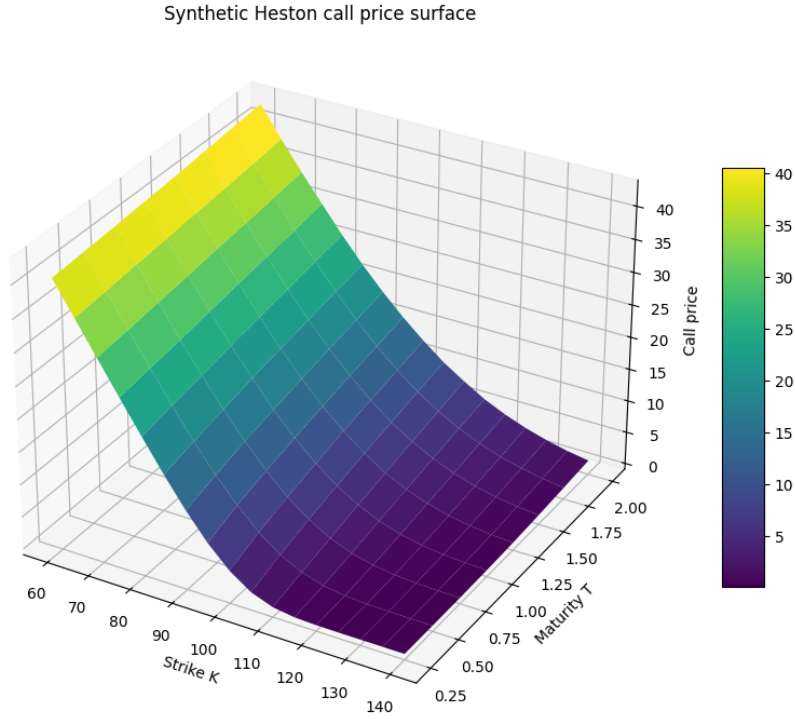


Figure 6: Synthetic Heston call price surface over the strike–maturity grid (K, T) , generated with the fixed parameter set $(S_0^*, Y_0^*, \kappa^*, \mu^*, \sigma^*, r^*, \rho^*)$. As expected for European call options, prices are decreasing in the strike and increasing in the maturity.

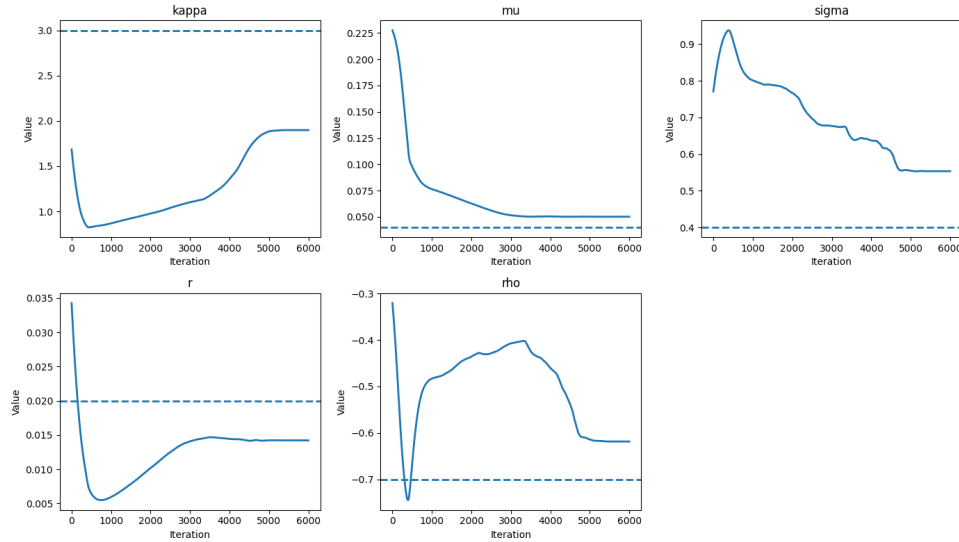


Figure 7: Evolution of calibrated parameters during optimization. Dashed lines indicate true parameter values.

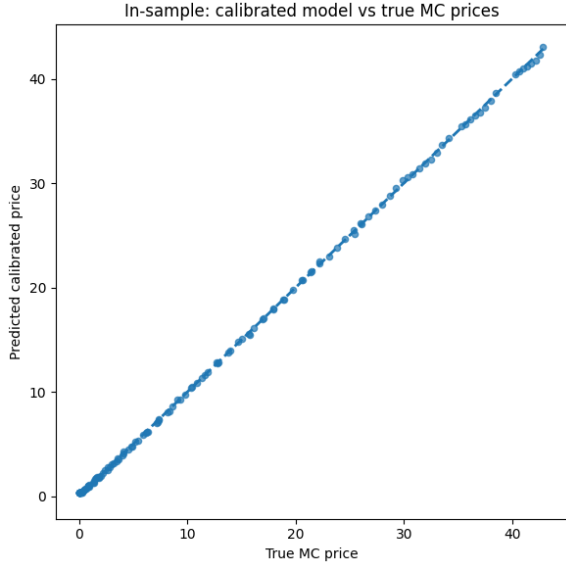


Figure 8: In-sample predicted vs true prices. Points lie close to the diagonal, indicating a good fit.

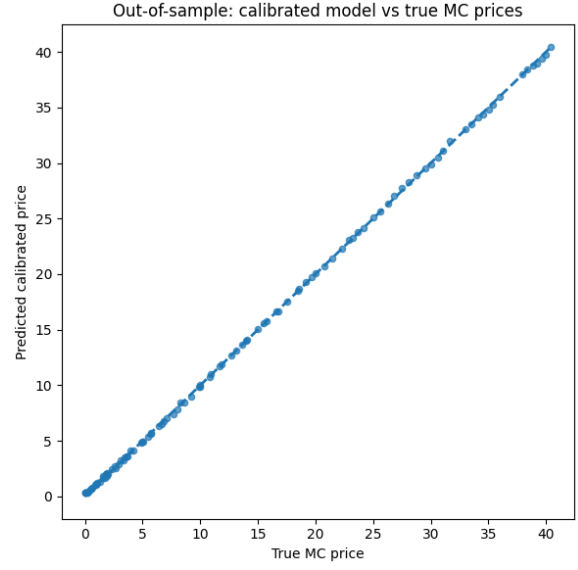


Figure 9: Out-of-sample predicted vs true prices. Points remain close to the diagonal, showing good generalization.

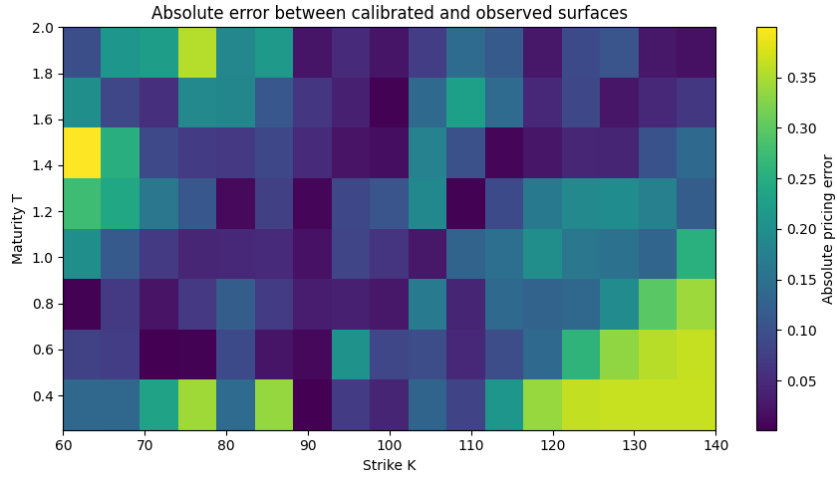


Figure 10: Absolute error between the calibrated neural-network surface and the synthetic observed surface across the strike–maturity grid (K, T) . The largest discrepancies are concentrated in some edge regions of the grid (especially extreme strikes). This is consistent with the fact that such contracts are more challenging to approximate and calibrate accurately.